PawSafe

Design Document — Version 1

BIL 481 — Software Engineering
Completion Date: February 25, 2026

List of Contributors

#	Name	Role
1	Yusuf Yanki Utgun	Team Lead / Frontend Developer
2	Eren Can Donertaş	Backend Developer
3	Enes Kerem Goksu	Full Stack Developer / QA
4	Yigit Eren Yilmaz	UI Designer / Tester

Table of Contents

Task Matrix

Section	Utgun	Donertaş	Goksu	Yilmaz
System Overview	Lead	Review	Contribute	Review
Implementation Details	Contribute	Lead	Contribute	Review
UML Diagrams	Lead	Contribute	Review	Contribute
Use Case Support	Review	Contribute	Lead	Contribute
Design Decisions	Contribute	Review	Contribute	Lead

1. System Overview

1.1 Brief Project Description

PawSafe is a community-driven web application where citizens can report dangerous stray animals on an interactive map. Users place location markers, upload photos as evidence, and validate each other's reports. A trust score system rewards reliable reporters and an admin panel provides moderation capability.

1.2 System Architecture Overview

PawSafe uses a Layered (N-Tier) Architecture with three layers: Presentation (TypeScript UI components), Business Logic (service classes), and Data Access (repository classes). This separation ensures testability and allows each layer to evolve independently. The frontend communicates with the backend via a REST API over HTTPS.

1.3 Technology Stack

Layer	Technology	Reason
Frontend	TypeScript + HTML5/CSS3	Type safety, team familiarity
Map	Leaflet.js 1.x (CDN)	Open-source, lightweight (~40 KB)
Backend	Node.js + Express (TS)	Same language as frontend, fast I/O
Database	PostgreSQL 15	Relational integrity, open-source
Auth	JWT (JSON Web Tokens)	Stateless, simple to implement
Image Storage	Local fs / AWS S3	Flexible for dev and production
Testing	Jest + Supertest	Standard Node.js testing stack

2. Implementation Details

2.1 Codebase Structure

- src/presentation/ — UI components: MapView, NotificationPanel, MarkerList, LeaderboardView, AdminPanel.
- src/business/ — Service classes: MarkerService, ValidationService, TrustScoreService, NotificationService, AuthService, ImageUploadService.
- src/data/ — Repository classes: MarkerRepository, UserRepository, ImageRepository, ValidationRepository.
- src/api/ — Express route handlers: markers.ts, users.ts, validations.ts.
- src/interfaces/ — Shared TypeScript interfaces and domain types.
- public/ — Static assets served to the browser.

2.2 Key Components and Modules

MapView

The main UI component. It initializes Leaflet.js, renders all existing markers on load, and handles map click events to trigger the marker creation flow. It communicates with MarkerService for CRUD operations.

MarkerService

Core business logic for marker management. Handles creation, retrieval, and deletion of markers. Validates input (coordinates, image size), calls MarkerRepository for persistence, and triggers NotificationService upon successful creation.

ValidationService

Manages community votes (validate/dispute) on markers. Prevents duplicate votes, persists votes via ValidationRepository, and updates the reporter's trust score via TrustScoreService after each validation.

2.3 Component Interfaces

Component	Key Methods	Description
MarkerService	createMarker(lat, lng, img, userId): Marker	Creates and saves a new marker.
MarkerService	getAll(): Marker[]	Returns all active markers.
ValidationService	validate(markerId, userId): boolean	Records validation, updates trust score.
TrustScoreService	increment(userId): void	Increases user trust score by 1.
AuthService	register(email, pw): User	Creates new user with hashed password.
AuthService	login(email, pw): string	Returns JWT on success.
ImageUploadService	upload(file: File): Promise<string>	Saves image, returns URL.
AdminService	removeMarker(markerId, adminId): void	Admin-only deletion of marker.

2.4 Visual Interfaces (UI Descriptions)

- Map Page — Full-screen Leaflet.js map. Markers shown as colored pins. Toolbar at top with Login/Register buttons. Click anywhere to place marker (if logged in).
- Location Info Card — Modal/popup showing: marker image, animal count, report date, validation count, and Validate / Dispute buttons.

- Notification Card — Toast notification in the top-right corner confirming actions (green for success, red for error). Auto-dismisses after 5 seconds.
- Leaderboard Page — Ranked list showing username, avatar, and trust score. Updated reactively when validations occur.
- Admin Panel — Table listing all markers with columns: ID, location, reporter, date, validation count. Each row has a Remove button.

3. UML Diagrams

3.1 Component Diagram

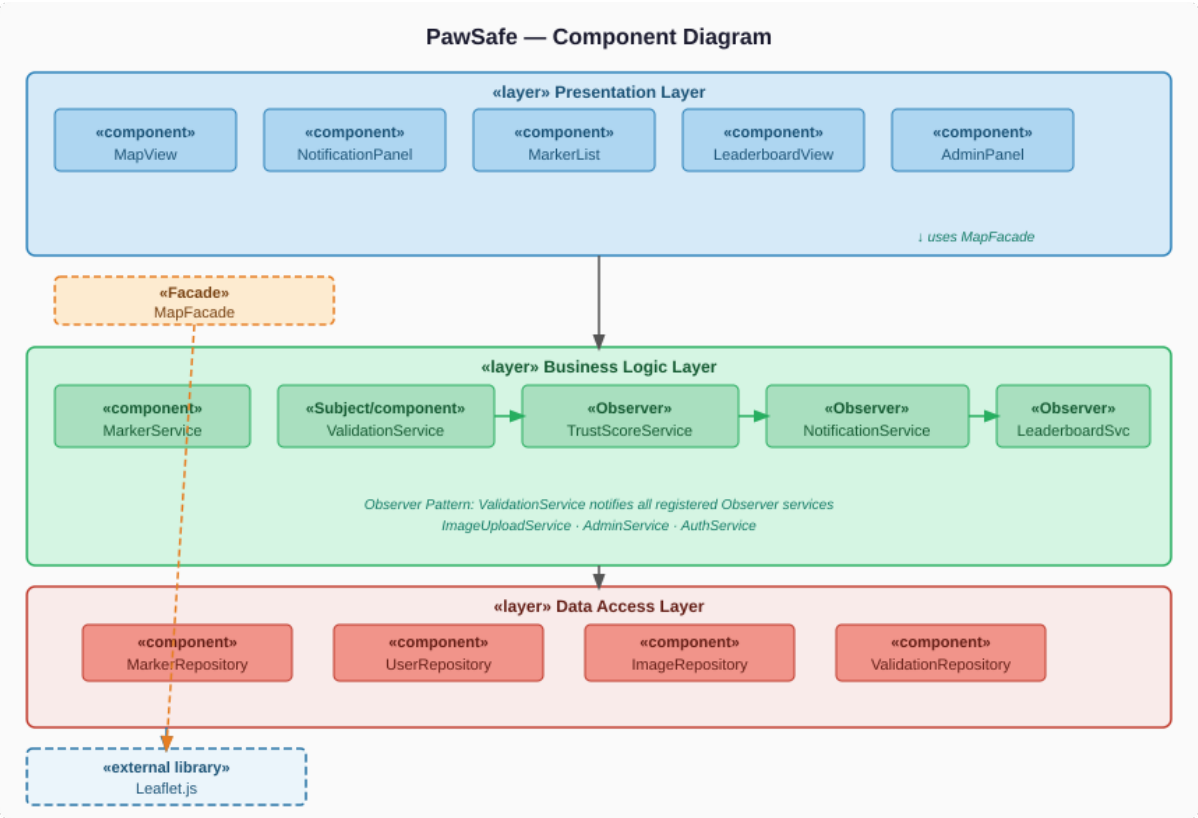

Figure 1 — Component Diagram: Layered Architecture showing Presentation, Business Logic, and Data Access layers.

3.2 Class Diagram

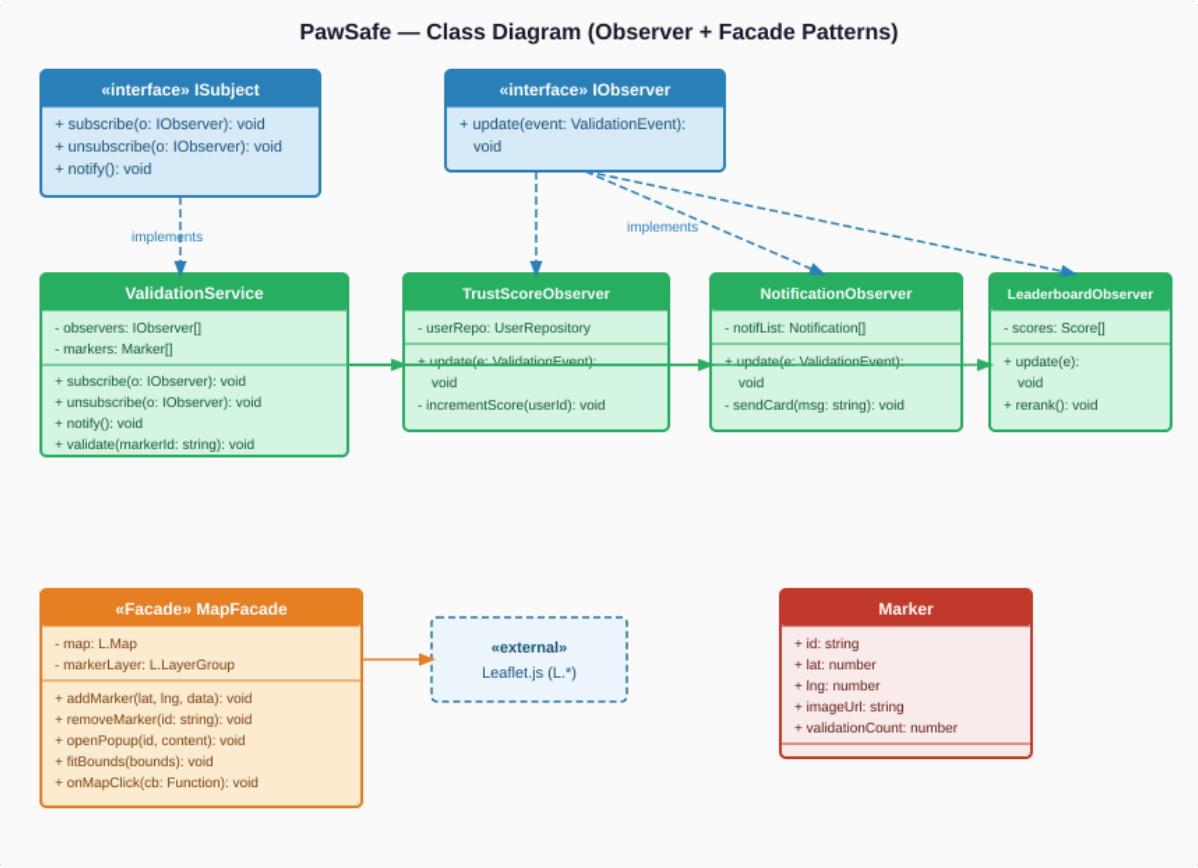

Figure 2 — Class Diagram: Key classes and interfaces in the PawSafe system.

3.3 Sequence Diagram

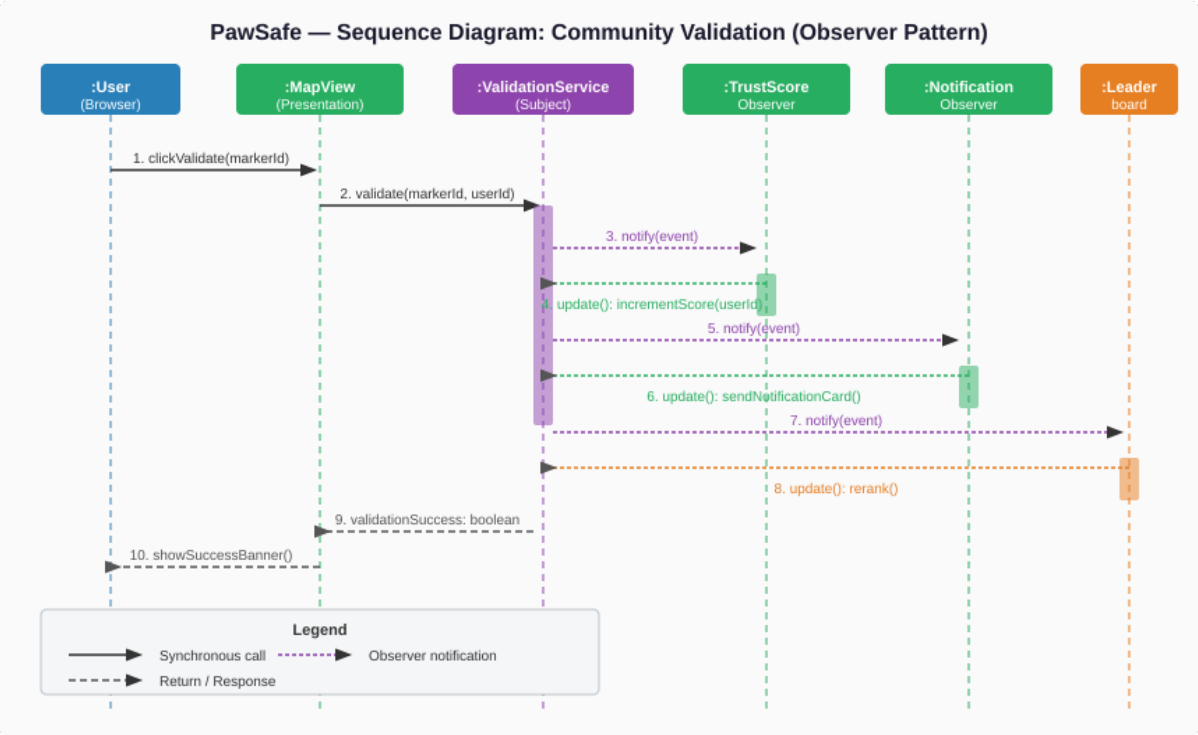

Figure 3 — Sequence Diagram: Community validation flow showing interactions between components.

3.4 Deployment Diagram

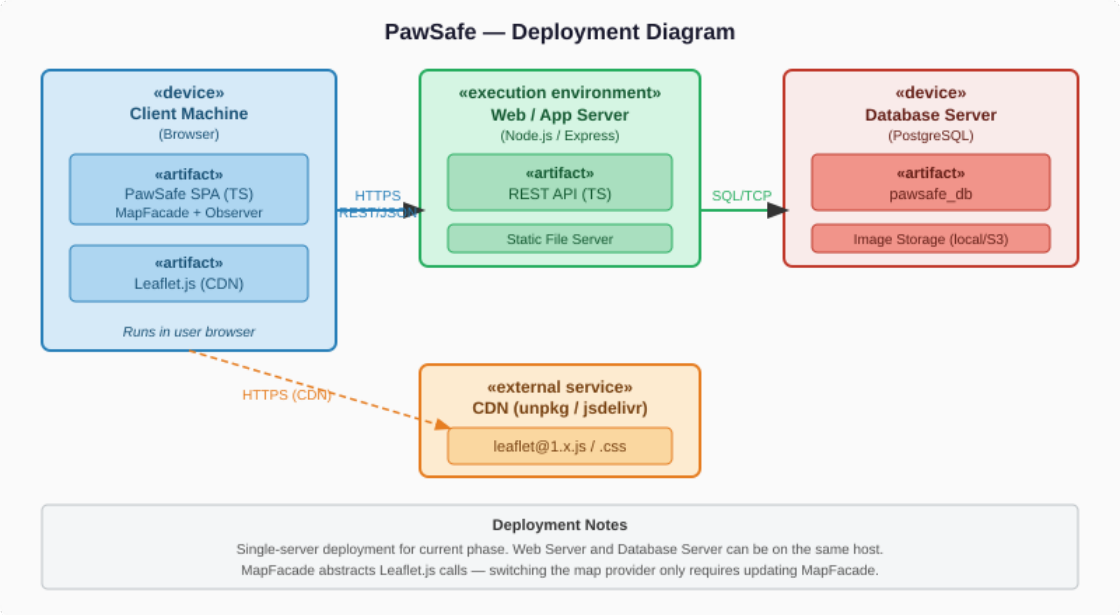

Figure 4 — Deployment Diagram: Single-server deployment model for PawSafe.

4. Use Case Support in Design

4.1 UC1 — Placing a Marker (FR-01, FR-02, FR-03)

- The user clicks the map → MapView's click event fires → User fills out the marker form (description + image) → ImageUploadService.upload() stores the photo → MarkerService.createMarker() persists the marker → MarkerRepository writes to PostgreSQL → NotificationService.show() triggers a notification card.
- Architecture support: Presentation Layer handles UI events; Business Layer handles all logic; Data Layer handles persistence. Clean separation.

4.2 UC2 — Community Validation (FR-06, FR-07, FR-08)

- User opens a Location Info Card and clicks Validate → MapView calls ValidationService.validate(markerId, userId) → Duplicate vote check performed → Vote saved via ValidationRepository → TrustScoreService.increment(reporterId) called → LeaderboardService updates rankings.
- Architecture support: ValidationService coordinates multiple services in the Business Logic Layer while remaining decoupled from the Presentation Layer.

4.3 UC3 — Viewing the Leaderboard (FR-08)

- User navigates to the Leaderboard page → LeaderboardView calls GET /api/leaderboard → Backend queries UserRepository ordered by trust_score DESC → Sorted list returned as JSON → LeaderboardView renders the ranking table.

4.4 UC4 — Admin Marker Removal (FR-09, FR-10)

- Admin logs in → AdminPanel calls GET /api/admin/markers (requires admin JWT) → Table of all markers rendered → Admin clicks Remove on a marker → AdminService.removeMarker() called → MarkerRepository.delete() removes from DB → MapView refreshes to remove the marker pin.

5. Design Decisions

5.1 Technology Comparison: Map Library

Criterion	Leaflet.js	Google Maps API
Cost	Free, open-source	Paid above free tier
Bundle size	~40 KB	~200 KB+
Customization	High (plugin ecosystem)	Moderate (proprietary)
Decision	Selected	Not selected

5.2 Technology Comparison: Backend Framework

Criterion	Express.js (Node/TS)	Django (Python)
Language consistency	Same as frontend (TS)	Requires Python knowledge
Performance	High (event-driven I/O)	Moderate (WSGI)
Team familiarity	High	Low
Decision	Selected	Not selected

5.3 Justification Summary

- PostgreSQL over MongoDB: relational data model (markers -> users -> validations) requires referential integrity. PostgreSQL's foreign key constraints prevent orphaned validation records.
- Layered Architecture over Microservices: appropriate for team size and project scope. Microservices would add unnecessary deployment complexity for a single-team academic project.
- JWT over session-based auth: stateless tokens work well with REST APIs and require no server-side session store.